

Bezpečnost

1 Teorie

Základem bezpečnosti je soubor security.yaml

```
# security.yaml

security:
  encoders:
    App\Entity\User: bcrypt

  providers:
    app_user_provider:
      entity:
        class: App\Entity\User
        property: email

  firewalls:
    main:
      anonymous: ~
      form_login:
        login_path: login
        check_path: login
        default_target_path: dashboard
        username_parameter: 'login_form[username]'
        password_parameter: 'login_form[password]'

  access_control:
    - { path: ^/dashboard, roles: ROLE_USER }

  role_hierarchy:
    ROLE_ADMIN: ROLE_USER
```

- encoders: Určuje, jakým způsobem jsou hesla šifrována. V tomto případě se používá bcrypt pro entitu App\Entity\User.
- providers: Definuje poskytovatele uživatelů. V tomto případě se využívá entita User a jako unikátní identifikátor uživatele se používá e-mailová adresa.
- firewalls: Zde se nastavují různé zabezpečené oblasti aplikace. V tomto případě je nastavený main firewall, který umožňuje anonymní přístup a formou přihlášení pomocí form_login.
- access_control: Definuje pravidla pro přístup k různým částem aplikace. V tomto případě je omezen přístup na URL /dashboard pouze pro uživatele s rolí ROLE_USER.

1.1.1 role_hierarchy

Definuje hierarchii rolí. Zde je ROLE_ADMIN přiřazena roli ROLE_USER.

```
# config/packages/security.yaml
security:
    # ...

    role_hierarchy:
        ROLE_ADMIN:    ROLE_USER
        ROLE_SUPER_ADMIN: [ROLE_ADMIN, ROLE_ALLOWED_TO_SWITCH]
```

Uživatelé s ROLE_ADMIN rolí budou mít také ROLE_USER roli.

Uživatelé s ROLE_SUPER_ADMIN, budou mít automaticky ROLE_ADMIN, ROLE_ALLOWED_TO_SWITCH a ROLE_USER (zděděno z ROLE_ADMIN).

1.2 Zabezpečení vzorů adres URL (access_control)

Nejzákladnějším způsobem, jak zabezpečit část vaší aplikace, je zabezpečit celý vzor adresy URL v security.yaml. Chcete-li například vyžadovat ROLE_ADMIN pro všechny adresy URL začínající na /admin, můžete:

```
# config/packages/security.yaml
security:
    # ...

    firewalls:
        # ...
        main:
            # ...

    access_control:
        # require ROLE_ADMIN for /admin*
        - { path: '^/admin', roles: ROLE_ADMIN }

        # or require ROLE_ADMIN or IS_AUTHENTICATED_FULLY for /admin*
        - { path: '^/admin', roles: [IS_AUTHENTICATED_FULLY, ROLE_ADMIN] }
```

1.2.1 Nastavení více rolí pro jednu routu

```
- { path: ^/admin, roles: [ROLE_ADMIN, ROLE_TEACHER] }
```

Pokud je uživatel přihlášen, ale nemá roli ROLE_ADMIN, zobrazí se mu stránka 403 přístup odepřen.

Dalším způsobem, jak zabezpečit jednu nebo více akcí ovladače, je použití #[IsGranted()] atributu.

```
// src/Controller/AdminController.php
// ...

use Symfony\Component\Security\Http\Attribute\IsGranted;

#[IsGranted('ROLE_ADMIN')]
class AdminController extends AbstractController
{
    // Optionally, you can set a custom message that will be displayed to the user
    #[IsGranted('ROLE_SUPER_ADMIN', message: 'You are not allowed to access the admin
dashboard.')]
    public function adminDashboard(): Response
    {
        // ...
    }
}

# config/packages/security.yaml
security:
    # ...

    access_control:
        # matches /admin/users/*
        - { path: '^/admin/users', roles: ROLE_SUPER_ADMIN }

        # matches /admin/* except for anything matching the above rule
        - { path: '^/admin', roles: ROLE_ADMIN }
```

Přidání cesty před ^ znamená, že se shodují pouze adresy URL *začínající* vzorem.

Například cesta /admin(bez ^) by odpovídala /admin/foo, ale také by odpovídala adresám URL jako /foo/admin.

Každý access_control se může také shodovat podle IP adresy, názvu hostitele a metod HTTP. Lze jej také použít k přesměrování uživatele na httpsverzi vzoru adresy URL. Pro složitější potřeby můžete využít i službu implementující RequestMatcherInterface.

1.3 Řízení přístupu v šablonách

Pokud chcete zkontrolovat, zda má aktuální uživatel určitou roli, můžete použít vestavěnou is_granted() pomocnou funkci v jakékoli šabloně Twig:

```
{% if is_granted('ROLE_ADMIN') %}
    <a href="#">Delete</a>
{% endif %}
```

1.4 Povolení nezabezpečeného přístupu (tj. anonymní uživatelé)

Když návštěvník ještě není přihlášen k vašemu webu, je považován za „neověřený“ a nemá žádné role. To jim zablokuje návštěvu vašich stránek, pokud jste definovali access_control pravidlo.

V access_control konfiguraci můžete pomocí PUBLIC_ACCESS atributu zabezpečení vyloučit některé cesty pro neověřený přístup (např. přihlašovací stránka):

```
# config/packages/security.yaml
security:

    # ...
    access_control:
        # allow unauthenticated users to access the login form
        - { path: ^/admin/login, roles: PUBLIC_ACCESS }

        # but require authentication for all other admin routes
        - { path: ^/admin, roles: ROLE_ADMIN }
```

1.5 Nastavení práv v šabloně

V šabloně Twig je uživatelský objekt dostupný prostřednictvím app.user proměnné.

```
{% if is_granted('IS_AUTHENTICATED_FULLY') %}
    <p>Email: {{ app.user.email }}</p>
{% endif %}
```

Zda je uživatel přihlášen

V šabloně

```
{% if app.user %}
    # user is logged in
{% else %}
    # user is not logged in
{% endif %}
```

Výpis loginu v šabloně

```
Email uživatele je: {{ app.user.email }}
```

Výpis role

```
{{ dump(app.user.roles) }}
```

2 Praktický návod na bezpečnost s users v db podle návodu z videa

<https://www.youtube.com/watch?v=8xP9YXZmjyc>

případně

<https://symfony.com/doc/current/security.html>

symfony new security --version="6.4.*" --webapp

Toto není nutné, jedině pokud potřebujete bootstrap v symfony,...

```
(  
composer require symfony/webpack-encore-bundle  
npm install  
npm run dev  
npm run build
```

)

2.1 Instalace bezpečnosti

composer require symfony/security-bundle

Dále spustíme

composer require --dev symfony/maker-bundle

Pozn

maker-bundle přináší sadu makerů, které umožňují rychle a snadno generovat kód pro běžné úlohy při vývoji. Například umožňuje generovat kód pro nové entity, kontroléry, formuláře, migrace databáze a mnoho dalšího, což výrazně zrychluje vývoj aplikace.

2.1.1 Některé z hlavních maker v **maker-bundle**:

1. Entity Maker: Umožňuje snadno vytvářet nové entity (entitní třídy) pro ORM (Object-Relational Mapping), jako je například Doctrine. Generuje základní třídy pro práci s databází.
2. Controller Maker: Pomáhá generovat základní kontroléry pro správu různých částí vaší aplikace.
3. Form Maker: Umožňuje vytvářet formuláře Symfony, což výrazně zjednodušuje proces vytváření a správy formulářů.
4. Migration Maker: Pomáhá generovat migrační soubory pro aktualizaci databáze na základě změn v entitách.
5. Messenger Maker: Pomáhá vytvářet komponenty zpracovávající asynchronní zprávy (např. zprávy v pozadí) v aplikaci.
6. Test Maker: Umožňuje generovat základní testy (jednotkové testy, integrační testy) pro ověření správnosti funkcionality vaší aplikace.

Po instalaci se také ve složce packages vytvoří soubor security.yaml

Nastavíme databázi v souboru .env

```
DATABASE_URL="mysql://pokus:pokus@127.0.0.1:3306/pokus?serverVersion=mariadb-10.4.28"
```

Vygenerujeme entitu User

symfony console make:user User

Aktualizujeme v databázi

symfony console make:migration

symfony console doctrine:migration:migrate

Instalace nástroje, který umožňuje implementaci ověření e-mailu v aplikaci. Tento balíček usnadňuje proces ověření platnosti e-mailových adres uživatelů.

composer require symfonycasts/verify-email-bundle

Po instalaci můžete využívat poskytované funkce, jako je například ověřování platnosti e-mailových adres uživatelů, odesílání ověřovacích e-mailů, správa životnosti ověřovacích tokenů atd.

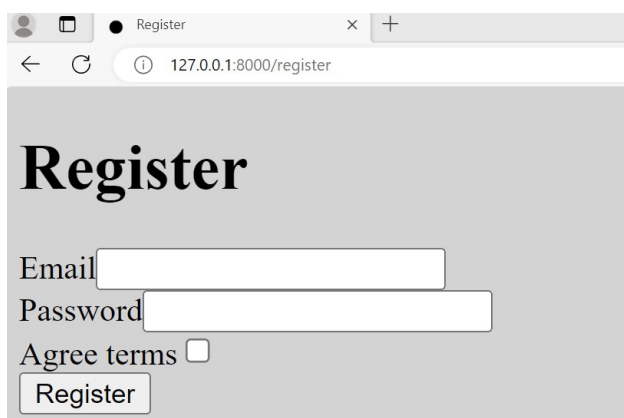
Po zadání těchto informací MakerBundle automaticky vygeneruje několik souborů:

1. Formulářová třída: Třída, která obsahuje konfiguraci formuláře v Symfony. Bude obsahovat definice polí formuláře, validační pravidla a manipulace s daty.
2. Šablona formuláře: Twig šablona, která slouží k vizuálnímu zobrazení formuláře v uživatelském rozhraní.
3. Controller: Základní kontrolér, který obsahuje akce pro zobrazení a odeslání formuláře.

Vytvoříme registrační formulář

symfony console make:registration-form

(Postupně vyplňujeme duplicity yes, sent email: no, automatic authenticate: yes, pak napsat _priview_error, unit test: no)



V RegistrationControlleru přidat routu `('/', name: 'main')`

```
class RegistrationController extends AbstractController  
{
```

```
#[Route('/', name: 'main')]
public function main(): Response
{
    return $this->render('base.html.twig');
}

#[Route('/register', name: 'app_register')]
public function register(Request $request,
```

A zde ještě opravit redirect na main

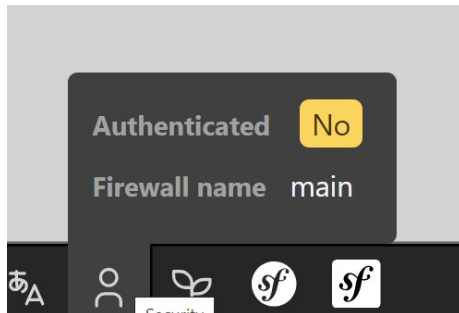
```
$entityManager->persist($user);
$entityManager->flush();
// do anything else you need here, like send an email

return $this->redirectToRoute('main');
```

Do base.html.twig přidat např. ahoj světe

```
{% block body %}
    <h1>ahoj svete</h1>
{% endblock %}
```

- nainstalovat (není nutné, už je nainstalovaný)
composer require --dev symfony/profiler-pack



V něm je vidět, že zatím zde je neautetifikovný user

Vytvoříme nový Login controller

php bin/console make:controller Login

Login controller opravíme podle

<https://symfony.com/doc/current/security.html>

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Component\Security\Http\Authentication\AuthenticationUtils;

class LoginController extends AbstractController
{
    #[Route('/login', name: 'app_login')]
    public function login(): Response
    {
        // ...
    }
}
```

```

    public function index(AuthenticationUtils $authenticationUtils):
Response
{
    // get the login error if there is one
    $error = $authenticationUtils->getLastAuthenticationError();

    // last username entered by the user
    $lastUsername = $authenticationUtils->getLastUsername();

    return $this->render('login/index.html.twig', [
        'controller_name' => 'LoginController',
        'last_username' => $lastUsername,
        'error' => $error,
    ]);
}
}

```

Také login/index.html.twig opravíme

```

{# templates/login/index.html.twig #}
{% extends 'base.html.twig' %}

{# ... #}

{% block body %}
    {% if error %}
        <div>{{ error.messageKey|trans(error.messageData,
'security') }}</div>
    {% endif %}

    <form action="{{ path('app_login') }}" method="post">
        <label for="username">Email:</label>
        <input type="text" id="username" name="_username"
value="{{ last_username }}">

        <label for="password">Password:</label>
        <input type="password" id="password" name="_password">

        {# If you want to control the URL the user is redirected to on
success
        <input type="hidden" name="_target_path" value="/account"> #}

        <button type="submit">login</button>
    </form>
{% endblock %}

```

Do security.yaml přidáme

```

main:
    lazy: true
    provider: app_user_provider
    form_login:
        login_path: app_login
        check_path: app_login

```

A

Zde je výpis celého security.yml

```
security:
    # https://symfony.com/doc/current/security.html#registering-the-user-
    hashing-passwords
    password_hashers:

Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
    'auto'

    # https://symfony.com/doc/current/security.html#loading-the-user-the-
    user-provider
    providers:
        # used to reload user from session & other features (e.g.
        switch_user)
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email

    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false

    main:
        lazy: true
        provider: app_user_provider
        form_login:
            login_path: app_login
            check_path: app_login
        logout:
            path: logout
            target: /

        # activate different ways to authenticate
        # https://symfony.com/doc/current/security.html#the-firewall

        #
        https://symfony.com/doc/current/security/impersonating_user.html
        # switch_user: true

        # Easy way to control access for large sections of your site
        # Note: Only the *first* access control that matches will be used
        access_control:
            # - { path: ^/admin, roles: ROLE_ADMIN }
            # - { path: ^/profile, roles: ROLE_USER }

when@test:
    security:
        password_hashers:
            # By default, password hashers are resource intensive and take
            time. This is
            # important to generate secure password hashes. In tests
            however, secure hashes
            # are not important, waste resources and increase test times.
            The following
            # reduces the work factor to the lowest possible values.

Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
    algorithm: auto
    cost: 4 # Lowest possible value for bcrypt
```

```
time_cost: 3 # Lowest possible value for argon
memory_cost: 10 # Lowest possible value for argon
```

Vysvětlení

1. lazy: true: Tato volba umožňuje lenivé načítání (lazy loading) firewallu. Pokud je nastaveno na true, firewall se bude načítat pouze v případě, že je skutečně vyvolán nějakým požadavkem na danou cestu, kterou chrání. To může zlepšit výkon aplikace tím, že firewall není inicializován, pokud není zapotřebí.
 2. provider: app_user_provider: Určuje, který poskytovatel bude použit pro autentizaci uživatelů. V tomto případě je app_user_provider poskytovatelem uživatelů, který pravděpodobně odkazuje na vaše vlastní nastavení uživatelského poskytovatele v konfiguraci Symfony.
 3. form_login: Definuje, jakým způsobem bude probíhat přihlašování pomocí formuláře.
- login_path: app_login: Toto je cesta, na kterou bude uživatel přesměrován, pokud není přihlášen a chce se dostat na stránku, která vyžaduje přihlášení.
 - check_path: app_login: Toto je cesta, na kterou bude odeslán formulář při pokusu o přihlášení. Symfony bude na této cestě očekávat POST požadavek s údaji uživatele pro ověření

A ještě přidáme řádek, kam se přepne po úspěšném přihlášení.

```
form_login:
    login_path: app_login
    check_path: app_login
    default_target_path: app_home
```

Přepne se po přihlášení na app_home

Start projektu

symfony server:start

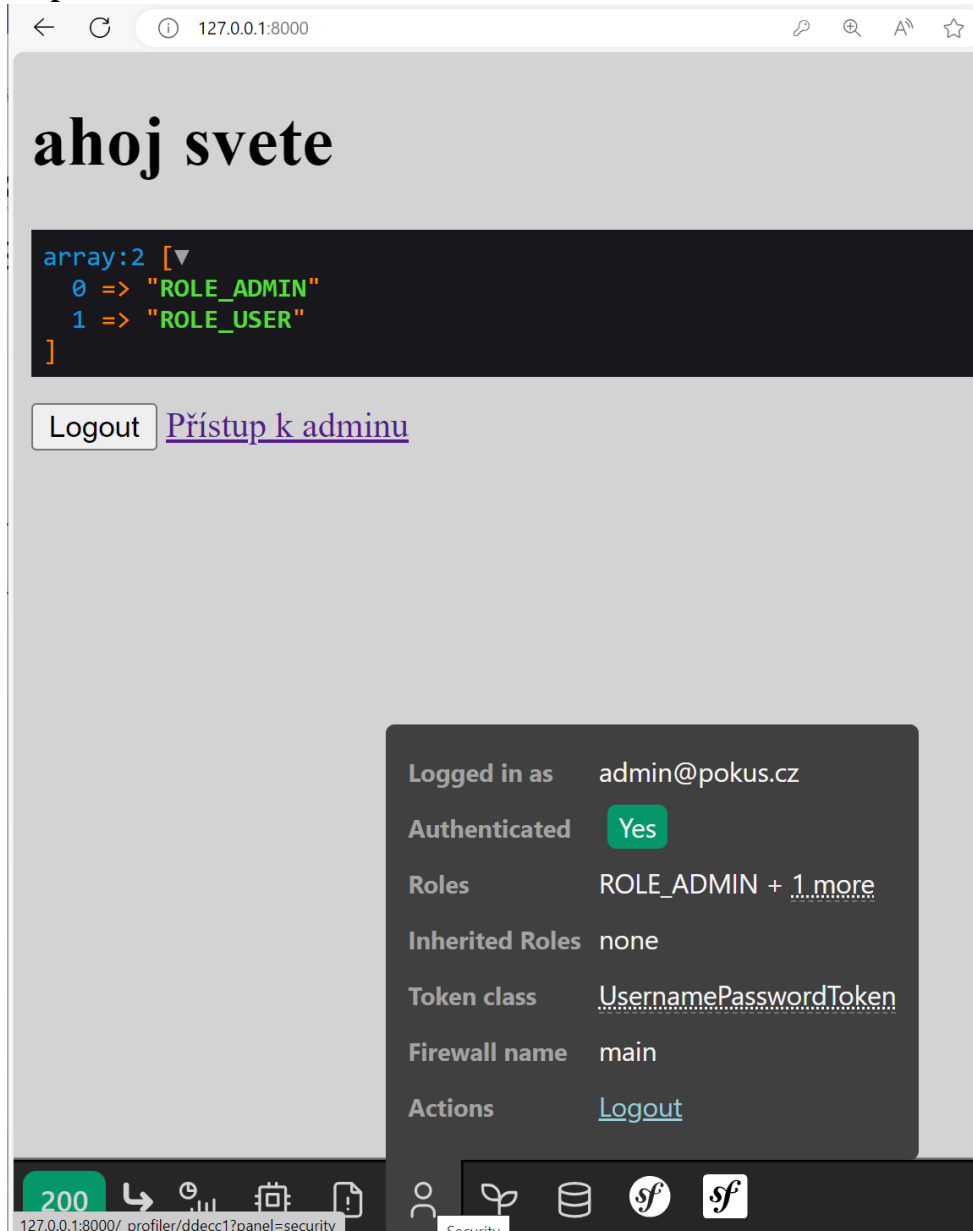
Provést registraci (vytvoření) uživatele přes webovou stránku **/register**

Po vytvoření uživatelů pomocí /register přidáme pomocí phpmyadminu role

!! nezapomenout úvozovky!!!

				id	email	roles (DC2Type=json)	password
☐	 Upravit	 Kopírovat	 Odstranit	1	superadmin@pokus.cz	["ROLE_SUPERADMIN"]	\$2y\$13\$0Uc4cbue14F
☐	 Upravit	 Kopírovat	 Odstranit	3	admin@pokus.cz	["ROLE_ADMIN"]	\$2y\$13\$ce0/Wc4qBhg
☐	 Upravit	 Kopírovat	 Odstranit	4	user@pokus.cz	[]	\$2y\$13\$cmgCdbCO1l

Po přihlášení



Do security.yaml přidat logout pro odhlášení

```
main:
  lazy: true
  provider: app_user_provider
  form_login:
    login_path: app_login
    check_path: app_login
  logout:
    path: app_logout
    target: /
```

Stačí dopsat main.

```
main:
  lazy: true
  provider: app_user_provider
  form_login:
    login_path: app_login
```

```
    check_path: app_login
  logout:
    path: logout
    target: /
```

a v config/routing.yaml přidat

```
controllers:
  resource:
    path: ../src/Controller/
    namespace: App\Controller
  type: attribute
app_logout:
  path: /logout
```

Vytvořit nový hlavní cotroller

symfony console make:controller HomeController

V něm vytvořit routu logout

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

class HomeController extends AbstractController
{
    #[Route('/logout', name: 'home')]
    public function index(): Response
    {
        return $this->render('home/index.html.twig', [
            'controller_name' => 'HomeController',
        ]);
    }
}
```

Pro jistotu zde je celý výpis

routing.yml

```
framework:
  router:
    utf8: true

    # Configure how to generate URLs in non-HTTP contexts, such as CLI
    # commands.
    # See https://symfony.com/doc/current/routing.html#generating-urls-
    # in-commands
```

```

#default_uri: http://localhost

when@prod:
    framework:
        router:
            strict_requirements: null

controllers:
    resource: ../src/Controller/
    type: attribute
    namespace: App\Controller

app_logout:
    path: /logout
    methods: GET

```

nebo takto, po odhlášení se přepne do login

```

controllers:
    resource:
        path: ../src/Controller/
        namespace: App\Controller
    type: attribute
app_login:
    path: /login

```

Do base.html.twig přidat tlačítko pro odhlášení

```

<button onclick="window.location='{{path('app_logout')}}'"> Logout
</button>

```

lépe

```

{% if app.user %}
<div>
<button onclick="window.location='{{path('app_logout')}}'"> Logout
</button>
    {% if is_granted('ROLE_ADMIN') %}
        <a href="{{ path('admin_img') }}">Přístup k adminu</a>
    {% endif %}
    {% if is_granted('ROLE_SUPERADMIN') %}
        <a href="{{ path('super_admin_img') }}">Přístup k superadminu</a>
    {% endif %}
{% endif %}
</div>
{% endblock %}

```

nedělat symfony console make:auth (1)

- V login controlleru nastavte přesměrování

Přidat do db role (pokud již nejsou)

				id	email	roles (DC2Type:json)	password			
☐		Upravit		Kopírovat		Odstranit	1	superadmin@pokus.cz	["ROLE_SUPERADMIN"]	\$2y\$13\$0Uc4cbue14F
☐		Upravit		Kopírovat		Odstranit	3	admin@pokus.cz	["ROLE_ADMIN"]	\$2y\$13\$ce0/Wc4qBhg
☐		Upravit		Kopírovat		Odstranit	4	user@pokus.cz	[]	\$2y\$13\$cmgCdbCO1l

Do security přidat práva na routy – stačí odblokovat #

```
- { path: ^/admin, roles: ROLE_ADMIN }
- { path: ^/admin/img, roles: ROLE_ADMIN }
```

případně `role_hierarchy`: - nedělat

```
role_hierarchy:
    ROLE_ADMIN: ROLE_USER
    ROLE_SUPER_ADMIN: ROLE_ADMIN
```

```
# Easy way to control access for large sections of your site
# Note: Only the *first* access control that matches will be used
access_control:
```

Vytvoříme MyController

php bin/console make:controller MyController

a upravíme obsah Controlleru

není nutné dělat

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class MyController extends AbstractController
{
    #[Route('/admin/img', name: 'admin_img')]
    public function adminImg()
    {
        //$file = new File('../public/img/admin.jpg');
        return new Response('jsem u admina');
        //return $this->file($file, "",
        ResponseHeaderBag::DISPOSITION_INLINE);
    }

    #[Route('/super-admin/img', name: 'super_admin_img')]
    public function superAdminImg()
    {
        //$file = new File('../public/img/super-admin.jpg');
        //$this->denyAccessUnlessGranted('ROLE_SUPERADMIN', null, 'User
        tried to access a page without having ROLE_ADMIN');
        return new Response('jsem u super admina');
        //return $this->file($file, "",
        ResponseHeaderBag::DISPOSITION_INLINE);
    }
}
```

Poznámka

Když je nastaveno v security.yaml anonymous: ~, znamená to, že určitá část aplikace nebude chráněna autentizačním mechanismem a bude volně dostupná pro uživatele, kteří nejsou přihlášení. To se často využívá pro veřejně dostupné stránky nebo sekce, které nevyžadují přihlášení, například úvodní stránky, kontaktní formuláře atd.

Zde je příklad security.yaml

```
access_control:
- { path: ^/login, roles: PUBLIC_ACCESS }
- { path: ^/admin, roles: ROLE_ADMIN }
- { path: ^/user, roles: ROLE_USER }
```

Původně bylo pro anonymního uživatele nastaveno:

```
- { path: ^/login, roles: IS_AUTHENTICATED_ANONYMOUSLY }
```

Od verze Symfony 5 anonymní uživatel neexistuje.

```
- { path: ^/login, roles: IS_AUTHENTICATED_ANONYMOUSLY }

access_control:
- { path: ^/login, roles: PUBLIC_ACCESS }
- { path: ^/admin, roles: ROLE_ADMIN }
- { path: ^/user, roles: ROLE_USER }
```

```
when@test:
    security:
        password_hashers:
            # By default, password hashers are resource intensive and take time. This is
            # important to generate secure password hashes. In tests however, secure hashes
            # are not important, waste resources and increase test times. The following
            # reduces the work factor to the lowest possible values.
            Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
                algorithm: auto
                cost: 4 # Lowest possible value for bcrypt
                time_cost: 3 # Lowest possible value for argon
                memory_cost: 10 # Lowest possible value for argon

        access_control:
            - { path: ^/admin, roles: ROLE_ADMIN }
            - { path: ^/profile, roles: ROLE_USER }
```

Další pokusy

```
- { path: ^/login, roles: PUBLIC_ACCESS }
- { path: ^/admin, roles: ROLE_ADMIN }
```

```
- { path: ^/user, roles: ROLE_USER }
- { path: ^/home, roles: ROLE_USER }
```

Po pokusu na routu bez přihlášení je uživatel přesměrován na login.

Výpis security.yml

```
security:
    # https://symfony.com/doc/current/security.html#registering-the-user-
    # hashing-passwords
    password_hashers:

Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
    'auto'
    # https://symfony.com/doc/current/security.html#loading-the-user-the-
    # user-provider
    providers:
        # used to reload user from session & other features (e.g.
        # switch_user)
        app_user_provider:
            entity:
                class: App\Entity\User
                property: email
    firewalls:
        dev:
            pattern: ^/(_(profiler|wdt)|css|images|js)/
            security: false
        main:
            lazy: true
            provider: app_user_provider
            form_login:
                login_path: app_login
                check_path: app_login
            logout:
                path: app_logout
                target: /

        # activate different ways to authenticate
        # https://symfony.com/doc/current/security.html#the-firewall

        #
        https://symfony.com/doc/current/security/impersonating_user.html
        # switch_user: true

        # Easy way to control access for large sections of your site
        # Note: Only the *first* access control that matches will be used

    access_control:
        - { path: ^/login, roles: PUBLIC_ACCESS }
        - { path: ^/admin, roles: ROLE_ADMIN }
        - { path: ^/user, roles: ROLE_USER }
        - { path: ^/home, roles: ROLE_USER }

when@test:
    security:
        password_hashers:
            # By default, password hashers are resource intensive and take
            # time. This is
```



```
        # important to generate secure password hashes. In tests
however, secure hashes
        # are not important, waste resources and increase test times.
The following
        # reduces the work factor to the lowest possible values.

Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
    algorithm: auto
    cost: 4 # Lowest possible value for bcrypt
    time_cost: 3 # Lowest possible value for argon
    memory_cost: 10 # Lowest possible value for argon
```